Credit Card Fraud Detection Using Machine Learning Classification Algorithms

MD. Siam Sarkar Faisal Navid Bin Salam Hasan Al Tanzim
Dept. of Computer Science & Engineering
BRAC University
Dhaka, Bangladesh
md.siam.sarkar@g.bracu.ac.bd faisal.navidbin.salam@g.bracu.ac.bd hasan.al.tanzim@g.bracu.ac.bd

Abstract—Credit card companies process a huge number of transactions every day, and checking each one by hand for fraud is not realistic. In this project we build and compare several machine learning classifiers that try to tell fraudulent credit card transactions apart from genuine ones. We use the Credit Card Fraud Detection Dataset 2023, which has 568,630 anonymized transactions from European cardholders and, unlike most fraud datasets, is exactly balanced between the two classes. After cleaning the data (removing duplicates, scaling the Amount feature, and doing an 80:20 stratified split), we trained six classifiers: Logistic Regression, Decision Tree, Random Forest, XGBoost, LightGBM, and an SGD-based Support Vector Machine. We evaluated all of them using Accuracy, Precision, Recall, F1-score, ROC-AUC, and confusion matrices. All six models did reasonably well on this balanced dataset, but the tree-based and boosting models clearly did better than the linear ones. XGBoost gave the best results overall, with an accuracy of 0.9997, an F1-score of 0.9997, and a ROC-AUC of 1.0000, with LightGBM and Random Forest close behind. Logistic Regression and the SGD-based SVM were weaker but still reached F1-scores above 0.96. Overall, the results suggest that ensemble and boosting tree methods work well for this task, though we also point out some limitations related to feature interpretability and the fact that the dataset is artificially balanced compared to real fraud rates.

Index Terms—credit card fraud detection, machine learning, classification, ensemble learning, XGBoost, LightGBM, Random Forest, imbalanced data

I. Introduction

A. Background

Online and card-based payments have grown enormously over the last decade, and this has meant a huge rise in the number of transactions banks and payment processors need to handle every day. This growth is good for customers and merchants, but it also opens the door to more fraud. Fraudulent transactions cost banks and cardholders money directly, and they also make people trust digital payments less. Since no team of people could manually check millions of transactions a day, automated systems that can flag suspicious activity are now a basic requirement for any payment provider.

Machine learning is one of the main tools used for this job. By learning patterns from past transaction data, a classifier can flag a suspicious transaction almost instantly, something a manual review process cannot match in speed or consistency. That said, fraud detection is not an easy classification problem. Real datasets are usually very imbalanced (fraud is rare), the features are often anonymized for privacy reasons, and there is a real tradeoff between missing actual fraud and generating too many false alarms. On top of this, fraud patterns can change over time as fraudsters adapt to whatever detection rules are currently in place, so a model that works well today is not guaranteed to keep working well indefinitely. This means that model evaluation has to go beyond a single accuracy number and look carefully at how a classifier behaves on the minority (fraud) class specifically, since that is the class that actually matters for the business problem.

B. Motivation

Even with all the existing research on this topic, it is still not obvious which algorithms work best under which conditions, how much preprocessing choices actually matter, or which evaluation metrics give the fairest picture of a model's usefulness for fraud detection. This project is our attempt to build a clear, reproducible pipeline: we take a real anonymized transaction dataset, preprocess it, train several classical and ensemble models on it, and evaluate them with metrics that make sense for a problem where both false positives and false negatives have real costs. We were also motivated by wanting to see this comparison done consistently, with the same preprocessing, the same train-test split, and the same evaluation metrics applied to every model, since many published comparisons use slightly different setups for each algorithm, which makes it harder to say whether one model is actually better or just tuned differently.

C. Problem Definition

The task addressed in this project is binary classification: given a transaction described by anonymized numerical features V_1–V_{28} and the transaction Amount, we want to predict whether the transaction is legitimate (Class 0) or fraudulent (Class 1). We compare six supervised classifiers – Logistic Regression, Decision Tree, Random Forest, XGBoost, LightGBM, and an SGD-trained Support Vector Machine – using Accuracy, Precision, Recall, F1-score, ROC-AUC, and confusion matrices, with the goal of finding which model is most suitable for this task.

II. Related Work

Fraud detection has been studied with many different algorithms, preprocessing choices, and ways of handling imbalance. Yilmaz [1] built a framework that combines Particle Swarm Optimization feature selection with several classifiers, including Artificial Neural Networks, Decision Tree, Logistic Regression, Naive Bayes, and Random Forest, and found that this optimized setup beat the individual baseline models on European cardholder data. Other work has combined ensemble methods with hybrid sampling strategies such as SMOTE and Edited Nearest Neighbor to deal with the class imbalance that is common in real fraud data [2]. Mim et al. [3] tried several balancing techniques (SMOTE, ADASYN, random undersampling) together with a soft voting ensemble and reported strong precision, recall, and AUROC.

Closer to our own dataset, Siam et al. [4] proposed a hybrid feature selection method combining Pearson correlation, Information Gain, and Random Forest Importance, and tested Random Forest, Extra Trees, XGBoost, AdaBoost, and CatBoost on the same European Cardholders 2023 dataset we use here, reaching up to 99.99% accuracy with Extra Trees. Other researchers have looked at density-based clustering combined with voting ensembles to improve recall on skewed data [5], built baseline comparisons across classical classifiers [6], and compared traditional classifiers with ensemble methods such as Random Forest, AdaBoost, and XGBoost using resampling and cross-validation [7]. Karunya et al. [8] looked at the trade-off between accuracy and computational cost for distance-based classifiers, while Al-Bulushi et al. [9] tested several imbalance-handling strategies together with bagging, boosting, and stacking across six datasets, finding that a bagging ensemble of Decision Tree, Random Forest, and an Artificial Neural Network gave the most consistent results.

Taken together, this earlier work points to the same general conclusion: ensemble and boosting tree methods (Random Forest, XGBoost, LightGBM, and similar algorithms) tend to beat simple linear classifiers on credit card fraud detection, and resampling or feature selection matters most when the dataset is imbalanced. This is what guided our choice of algorithms and evaluation approach.

Most of the studies above either work with an imbalanced dataset and therefore need to spend significant effort on resampling or feature selection, or they focus on only two or three algorithms at a time. Our project takes a slightly different angle: because the Credit Card Fraud Detection Dataset 2023 is already exactly balanced, we can isolate the comparison between six different classifiers (three commonly used baselines plus three ensemble/boosting methods) without the extra variable of a resampling strategy. This lets us look specifically at how much of the performance gap between linear and tree-based models comes from the algorithms themselves, separate from any imbalance-handling technique.

III. Dataset Description

We use the Credit Card Fraud Detection Dataset 2023, which contains anonymized transactions from European cardholders. It has 568,630 transactions across 31 columns: a transaction id, 28 anonymized numerical features (V_1–V_{28}), the transaction Amount, and the binary target Class (0 = legitimate, 1 = fraud). One thing that stands out about this dataset is that it is exactly balanced: 284,315 legitimate and 284,315 fraudulent transactions, a 50:50 split. Most real fraud datasets are highly imbalanced, so this is not typical, but it does mean we did not need resampling methods such as SMOTE. There were no missing values anywhere in the 31 columns, and every feature is numerical, so we did not need to do any categorical encoding either.

TABLE I
TRANSACTION AMOUNT SUMMARY BY CLASS

Class	Mean	Median	Std. Dev.	Max
Legitimate (0)	12,026.31	11,996.90	6,929.50	24,039.93
Fraud (1)	12,057.60	12,062.45	6,909.75	24,039.93

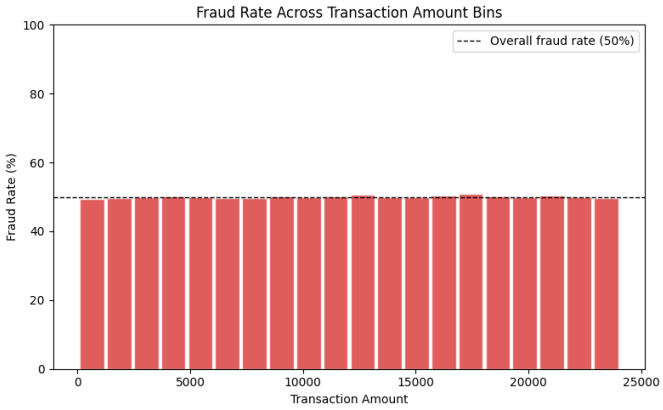

Fig. 1. Fraud rate across 20 equal-width transaction-amount bins. The dashed line marks the overall 50% fraud rate.

Because V_1–V_{28} are anonymized (likely produced by a transformation such as PCA on the original transaction fields), we cannot know what they actually represent in the real world. This limits how far we can interpret the results: even if a feature correlates strongly with fraud, we cannot say whether it reflects location, merchant type, time of transaction, or something else. Because of this, we treat V_1–V_{28} purely as numerical predictors and focus on predictive performance rather than trying to explain what each feature means. Amount is the only feature whose meaning is directly interpretable, ranging from about 50.01 to 24,039.93 with an overall mean of about 12,041.96 and a standard deviation of about 6,919.64.

Fig. 1 shows how the fraud rate behaves across the range of the Amount variable: we split transactions into 20 equal-width bins by Amount and computed the fraction of fraudulent transactions inside each bin. The fraud rate stays close to the overall 50% baseline in essentially every bin, which confirms that transaction amount by itself does not shift the odds of a transaction being fraudulent. Table I and the boxplot in Fig. 2 support this from a different angle: the mean and median Amount for legitimate and fraudulent transactions are nearly identical.

We also ran a Pearson correlation between each anonymized

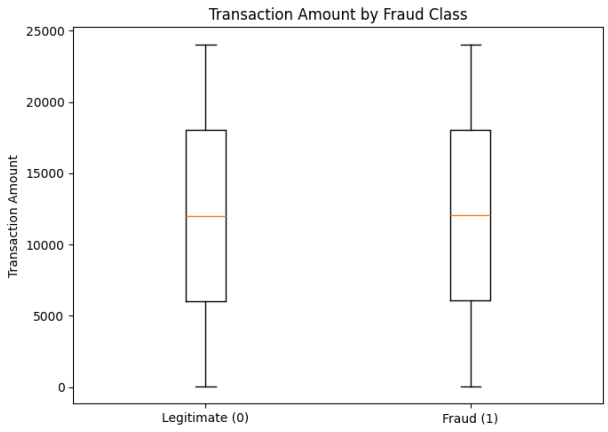

Fig. 2. Transaction Amount compared across the Legitimate and Fraud classes.

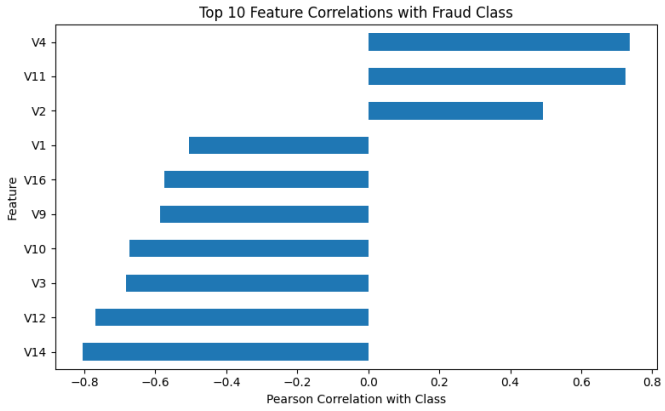

Fig. 3. Top 10 feature correlations with the fraud class label.

feature and Class. The strongest relationships were V_{14} ($r \approx -0.8057$), V_{12} ($r \approx -0.7686$), V_4 ($r \approx 0.7360$), V_{11} ($r \approx 0.7243$), V_3 ($r \approx -0.6821$), and V_{10} ($r \approx -0.6737$). This shows that several anonymized features on their own, or in combination, carry useful information for telling fraud apart from legitimate activity. Fig. 3 shows the top 10 features ranked by absolute correlation with the target.

IV. Methodology

A. Data Preprocessing

Before training any models, we applied the following preprocessing steps:

- **Removing the id column**: this column is only a unique transaction identifier, so it carries no useful predictive signal.
- **Duplicate removal**: we found one duplicate transaction (ignoring id) and removed it, which brought the dataset from 568,630 down to 568,629 rows.
- **Missing values**: there were none, so no imputation was needed.
- **Categorical encoding**: every predictor is already numerical, so no encoding step was required.
- **Class balance**: after cleaning, the split was still almost exactly 50:50 (about 50.0001% vs. 49.9999%), so we did not apply any oversampling or undersampling.
- **Train-test split**: we split the data into 80% training (454,903 transactions) and 20% testing (113,726 transactions) using stratified sampling so both sets kept the same class ratio, with `random_state=42` for reproducibility.
- **Feature scaling**: we standardized the Amount column with `StandardScaler`, fitting it only on the training data and then applying it to the test data to avoid leaking test information into training. The V_1–V_{28} features were already on comparable numerical scales, so we left them as they were.

Fig. 4 summarizes this preprocessing pipeline end to end, from the raw 568,630-row dataset down to the final model-ready training and testing matrices.

B. Models

We trained and compared six supervised classifiers:

- **Logistic Regression**: a simple linear baseline (`max_iter=1000`).
- **Decision Tree Classifier**: a single tree limited to a maximum depth of 15, which can capture nonlinear relationships between features.
- **Random Forest Classifier**: an ensemble of 25 decision trees (`max_depth=15`) combined through bootstrap aggregating.
- **XGBoost**: a gradient-boosted tree ensemble (`n_estimators=100`) where each new tree tries to correct the mistakes of the previous ones.
- **LightGBM**: a histogram-based gradient boosting model (`n_estimators=100`), chosen mainly because it trains efficiently on large tabular datasets like this one.
- **Support Vector Machine (SGD-based)**: implemented with `SGDClassifier` using log-loss, which approximates a linear SVM/logistic model trained through stochastic gradient descent, with `class_weight='balanced'`.

All models were trained with `random_state=42` for reproducibility. Hyperparameters were set to reasonable, commonly used starting values rather than being tuned through grid search: the tree-based models were capped at a maximum depth of 15 to limit overfitting, Random Forest used 25 trees as a balance between ensemble stability and training time, and the two boosting models used 100 estimators, a typical default for gradient boosting on tabular data of this size.

C. Tools Used

The project was implemented in Python. We used pandas and NumPy for data loading and manipulation; scikit-learn for Logistic Regression, Decision Tree, Random Forest, `SGDClassifier`, preprocessing utilities (`StandardScaler`, `ColumnTransformer`, `Pipeline`), and evaluation metrics; the dedicated `xgboost` and `lightgbm` libraries for the two gradient boosting models; and Matplotlib for all graphs. All experiments were run inside a Jupyter notebook.

TABLE II
PERFORMANCE COMPARISON OF CLASSIFICATION MODELS

Model	Acc.	Prec.	Rec.	F1	AUC
Logistic Regression	0.9648	0.9771	0.9519	0.9644	0.9935
Decision Tree	0.9956	0.9944	0.9968	0.9956	0.9972
Random Forest	0.9992	0.9993	0.9991	0.9992	1.0000
XGBoost	**0.9997**	0.9995	**1.0000**	**0.9997**	1.0000
LightGBM	0.9993	0.9985	**1.0000**	0.9993	0.9999
SVM (SGD)	0.9648	0.9763	0.9528	0.9644	0.9932

D. Evaluation Strategy

Both false positives (a legitimate transaction wrongly flagged as fraud) and false negatives (fraud that slips through) have real costs, so we did not rely on accuracy alone. We evaluated every model using Accuracy, Precision, Recall, F1-score, ROC-AUC, confusion matrices, and full classification reports. Treating fraud as the positive class and writing TP, TN, FP, FN for true positives, true negatives, false positives, and false negatives, the core metrics are defined as

$$\text{Precision} = \frac{TP}{TP + FP}, \tag{1}$$

$$\text{Recall} = \frac{TP}{TP + FN}, \tag{2}$$

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \tag{3}$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \tag{4}$$

Recall is arguably the most important of these for this problem, since a low recall means fraudulent transactions are slipping through undetected, while ROC-AUC summarizes how well a model separates the two classes across every possible decision threshold rather than only the default of 0.5.

V. Results and Evaluation

Table II shows how all six models performed on the held-out test set of 113,726 transactions (56,863 legitimate and 56,863 fraudulent).

XGBoost gave the strongest overall result, with an accuracy of 0.9997, an F1-score of 0.9997, a recall of 1.0000 (it caught every single fraud case in the test set), and a ROC-AUC of 1.0000. LightGBM was almost as good, also reaching perfect recall but with a slightly lower precision (0.9985), which means it produced a few more false positives than XGBoost. Random Forest was also very strong, with every metric above 0.999.

The Decision Tree did well too (F1-score of 0.9956), but it was clearly behind the ensemble and boosting models, which fits the general expectation that a single tree tends to overfit more than an ensemble of trees. Logistic Regression and the SGD-based SVM gave the weakest results of the six, though they were still respectable, with F1-scores around 0.964 and ROC-AUC above 0.99. Their lower recall values (0.9519 and 0.9528) show that these two linear models missed a larger share of fraud cases than the tree-based methods did. Table II summarizes these differences at a glance: the two linear models sit noticeably lower on every metric, while the ensemble and boosting models cluster near 1.0, with XGBoost and LightGBM slightly ahead due to their perfect recall.

Because XGBoost produced the strongest overall metrics, Fig. 5 shows its confusion matrix on the held-out test set: it correctly classified 56,832 of 56,863 legitimate transactions and all 56,863 fraud cases, with only 31 false positives and zero missed frauds. For context, LightGBM missed only 1 fraud case out of 56,863 and Random Forest missed 51, while Logistic Regression and the SGD-based SVM each missed more than 2,600 fraud cases – confirming the same pattern of ensemble and boosting models outperforming the two linear classifiers specifically on the class that matters most for fraud detection.

The ROC-AUC column in Table II tells a similar story: XGBoost and Random Forest both reach a perfect AUC of 1.0000 and LightGBM reaches 0.9999, while Logistic Regression and the SGD-based SVM trail at 0.9935 and 0.9932 respectively, consistent with their lower recall on the fraud class.

VI. Discussion

Our results reinforce the pattern seen throughout the literature reviewed in Section II: ensemble and boosting tree methods consistently outperform simple linear classifiers on credit card fraud data. Yilmaz [1] found that a PSO-optimized ensemble beat individual baselines, including Logistic Regression and Naive Bayes, on the same European cardholder population; our results show an even sharper gap, since our boosting models (XGBoost, LightGBM) reached near-perfect recall while Logistic Regression and the SGD-based SVM plateaued around 0.95. Assabil and Obagbuwa [7] and Al-Bulushi et al. [9] similarly reported that ensemble methods (Random Forest, AdaBoost, XGBoost, and bagged ensembles) outperformed Logistic Regression and Decision Tree baselines once resampling was applied to their imbalanced data; we observe the same ranking even though our dataset needed no resampling at all, since it is already balanced 50:50. The closest comparison is Siam et al. [4], who used the same Credit Card Fraud Detection Dataset 2023 and reported up to 99.99% accuracy with Extra Trees after a hybrid feature-selection pipeline combining Pearson correlation, Information Gain, and Random Forest Importance. Our best model, XGBoost, reached 99.97% accuracy without any feature-selection step (Table III), which suggests the anonymized features here are informative enough on their own that heavy feature engineering may not be strictly necessary for near-perfect results with tree-based models on this dataset. Ghalwash et al. [5] prioritized recall through a DBSCAN-augmented voting ensemble on heavily skewed data; our XGBoost model reaches a comparable ceiling (perfect recall) here, though on an already-balanced dataset rather than a skewed one, so the two results are not fully comparable. Karunya et al. [8] pointed out that computational cost matters as much as raw accuracy for real-time deployment, a trade-off our comparison does not address directly since we optimized purely for classification performance rather than inference latency.

Beyond the direct dataset overlap with Siam et al. [4], the rank-

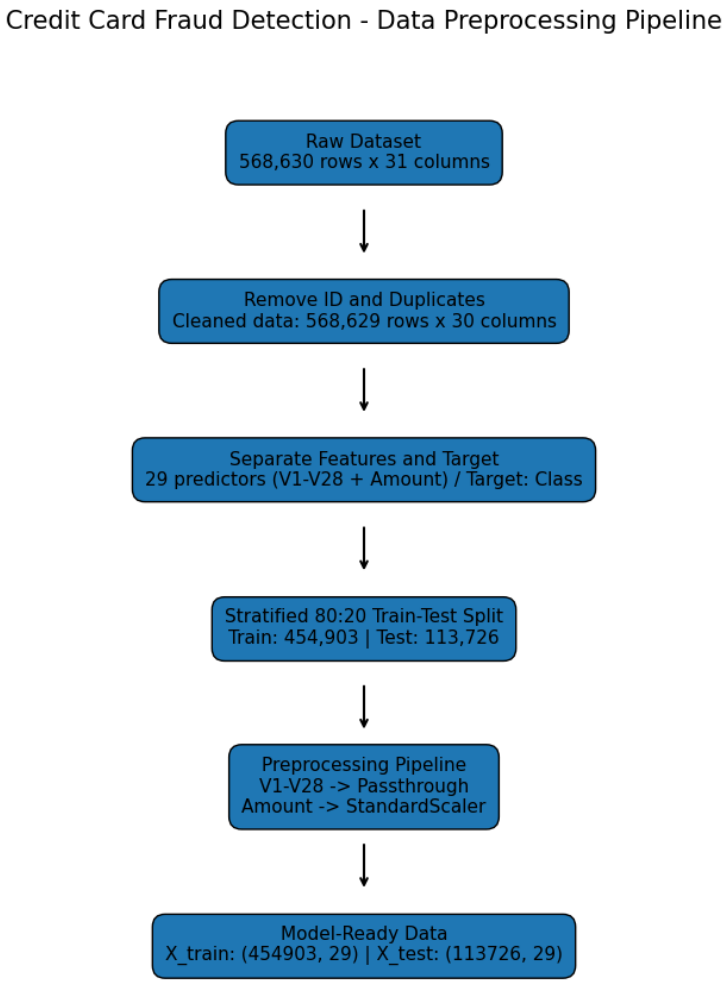

Fig. 4. Data preprocessing pipeline, from the raw dataset to the model-ready training and testing sets. Shown enlarged for readability.

TABLE III
COMPARISON WITH RELATED WORK ON SIMILAR DATA

Study	Best Model	Reported Result
Siam et al. [4]	Extra Trees	99.99% accuracy
Al-Bulushi et al. [9]	Bagging (DT+RF+ANN)	0.99 acc., 0.90 rec., 0.77 prec.
This work	XGBoost	0.9997 acc., 1.0000 rec., 0.9995 prec.

ing we observe is consistent with the broader set of studies covered in Section II. Mim et al. [3] combined several balancing techniques with a soft voting ensemble and reported strong precision, recall, and AUROC; our results reach a similar ceiling on precision and AUROC without needing any balancing step at all, since the Credit Card Fraud Detection Dataset 2023 is already balanced 50:50. Aslam and Hussain [6] built baseline comparisons across classical classifiers and found that specific classifiers could be clearly isolated as the strongest on their data; we observe the same kind of separation here, with XGBoost and LightGBM standing well apart from Logistic Regression and the SGD-based SVM on every metric in Table II. Looking at the per-class breakdown behind Table II also helps explain what is really happening with the two weaker models: Logistic Regression and the SGD-based SVM reach about 0.98 recall on the Legitimate class but only around 0.95 recall on the Fraud class, the opposite of what a bank would normally want to prioritize. This is the kind of asymmetry that would typically call for a lower decision threshold or explicit class weighting if a linear model had to be used in practice for speed or interpretability reasons, even if that means accepting a few more false alarms.

A few limitations should be kept in mind:

- **Artificial class balance**: the dataset is exactly balanced (50:50), unlike real transaction streams where fraud is typically under 1%, so precision-recall trade-offs would likely be harder to manage in production.
- **Limited explainability**: because V_1–V_{28} are anonymized PCA-style features, we cannot say which real transaction properties drive the strong correlations observed for V_{14}, V_{12}, or V_4, which limits explainability for deployment in a bank setting where this is often a legal requirement.

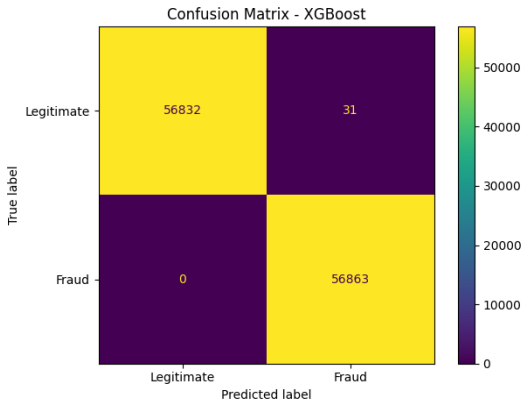

Fig. 5. Confusion matrix for the best-performing model (XGBoost) on the held-out test set of 113,726 transactions.

- **Single train-test split**: we relied on one stratified 80:20 split with default-style hyperparameters rather than repeated cross-validation and systematic tuning, so there is likely still some performance left on the table for the top models.
- **Static evaluation**: our test set is a snapshot rather than a stream, so we cannot yet say how well any of these models would hold up as fraud patterns drift over time, an issue raised in Section I.

VII. Conclusion

In this project we built and compared six machine learning classifiers – Logistic Regression, Decision Tree, Random Forest, XGBoost, LightGBM, and an SGD-based Support Vector Machine – for credit card fraud detection, using the Credit Card Fraud Detection Dataset 2023, a balanced set of 568,629 anonymized European cardholder transactions after cleaning. After preprocessing (duplicate removal, feature scaling, and an 80:20 stratified split), we evaluated every model with Accuracy, Precision, Recall, F1-score, ROC-AUC, and confusion matrices. XGBoost gave the best overall performance (Accuracy = 0.9997, F1 = 0.9997, ROC-AUC = 1.0000), with LightGBM and Random Forest close behind, while Logistic Regression and the SGD-based SVM were weaker but still reasonably strong. These results agree with the pattern seen in prior research: ensemble and gradient boosting tree methods tend to be well suited to this kind of fraud classification task.

For future work, we plan to test these models under more realistic, highly imbalanced class distributions, run proper hyperparameter tuning and cross-validation, and apply explainability techniques such as SHAP to better understand which anonymized features are actually driving the predictions, with the eventual goal of building a fraud detection pipeline that is both accurate and practical enough for real financial monitoring. It would also be worth testing distance-based methods such as k-nearest neighbors, which one comparative study on similar imbalanced data found to be the strongest performer once combined with resampling [7], even though our own dataset needed no resampling at all. Finally, we would like to test how well the models trained here generalize to a different fraud dataset rather than just a held-out split of the same one.

From a deployment perspective, even a model that reaches perfect recall in a controlled evaluation like this one would still need to be paired with a human review step and revisited periodically, since fraud patterns are known to drift as fraudsters adapt to whatever detection rules are currently in place. A production pipeline built on these results would therefore likely combine a strong classifier such as XGBoost with ongoing monitoring of its precision and recall on live traffic, rather than treating the offline test-set numbers reported here as a permanent guarantee of future performance. More broadly, by applying the same preprocessing, the same train-test split, and the same evaluation metrics across all six models, this project set out to give a fair, apples-to-apples comparison between classical and ensemble methods on this dataset – a level of consistency that, as noted in Section I-B, is not always guaranteed across the wider published literature.

References

[1] A. A. Yılmaz, "A machine learning-based framework using the particle swarm optimization algorithm for credit card fraud detection," *Communications Faculty of Sciences University of Ankara Series A2-A3 Physical Sciences and Engineering*, vol. 66, no. 1, pp. 82–94, 2024.

[2] "A credit card fraud detection approach based on ensemble machine learning classifier with hybrid data sampling," *Machine Learning with Applications*, vol. 20, p. 100675, 2025.

[3] M. A. Mim, N. Majadi, and P. Mazumder, "A soft voting ensemble learning approach for credit card fraud detection," *Heliyon*, vol. 10, no. 3, p. e25466, 2024.

[4] A. M. Siam, P. Bhowmik, and M. P. Uddin, "Hybrid feature selection framework for enhanced credit card fraud detection using machine learning models," *PLOS ONE*, vol. 20, no. 7, p. e0326975, 2025.

[5] M. A. Ghalwash, S. M. Abdelrazek, N. H. Eladawi, et al., "Enhancing credit card fraud detection using DBSCAN-augmented disjunctive voting ensemble," *Scientific Reports*, 2025. doi: 10.1038/s41598-025-22960-w.

[6] A. Aslam and A. Hussain, "A performance analysis of machine learning techniques for credit card fraud detection," *Journal on Artificial Intelligence*, vol. 6, no. 1, 2024.

[7] J. J. Assabil and I. C. Obagbuwa, "Credit card fraud detection using machine learning algorithms: A comparative study of six models," *International Journal of Intelligent Systems and Applications in Engineering*, vol. 12, no. 23s, pp. 862–875, 2024.

[8] R. V. Karunya, S. Ganesh, D. Muralidharan, G. R. Brindha, and M. Thiruvengadam, "Credit card fraud data analysis and prediction using machine learning algorithms," *Security and Privacy*, vol. 8, p. e70043, 2025.

[9] A. Al-Bulushi, A. K. Shaikh, and N. Adhikari, "Applying supervised machine learning algorithms and ensemble models to enhance credit card fraud detection," *Frontiers in Artificial Intelligence*, vol. 9, p. 1813728, 2026.

[10] Kaggle, "Credit Card Fraud Detection Dataset 2023." [Online]. Available: https://www.kaggle.com/datasets/nelgiriyewithana/credit-card-fraud-detection-dataset-2023